

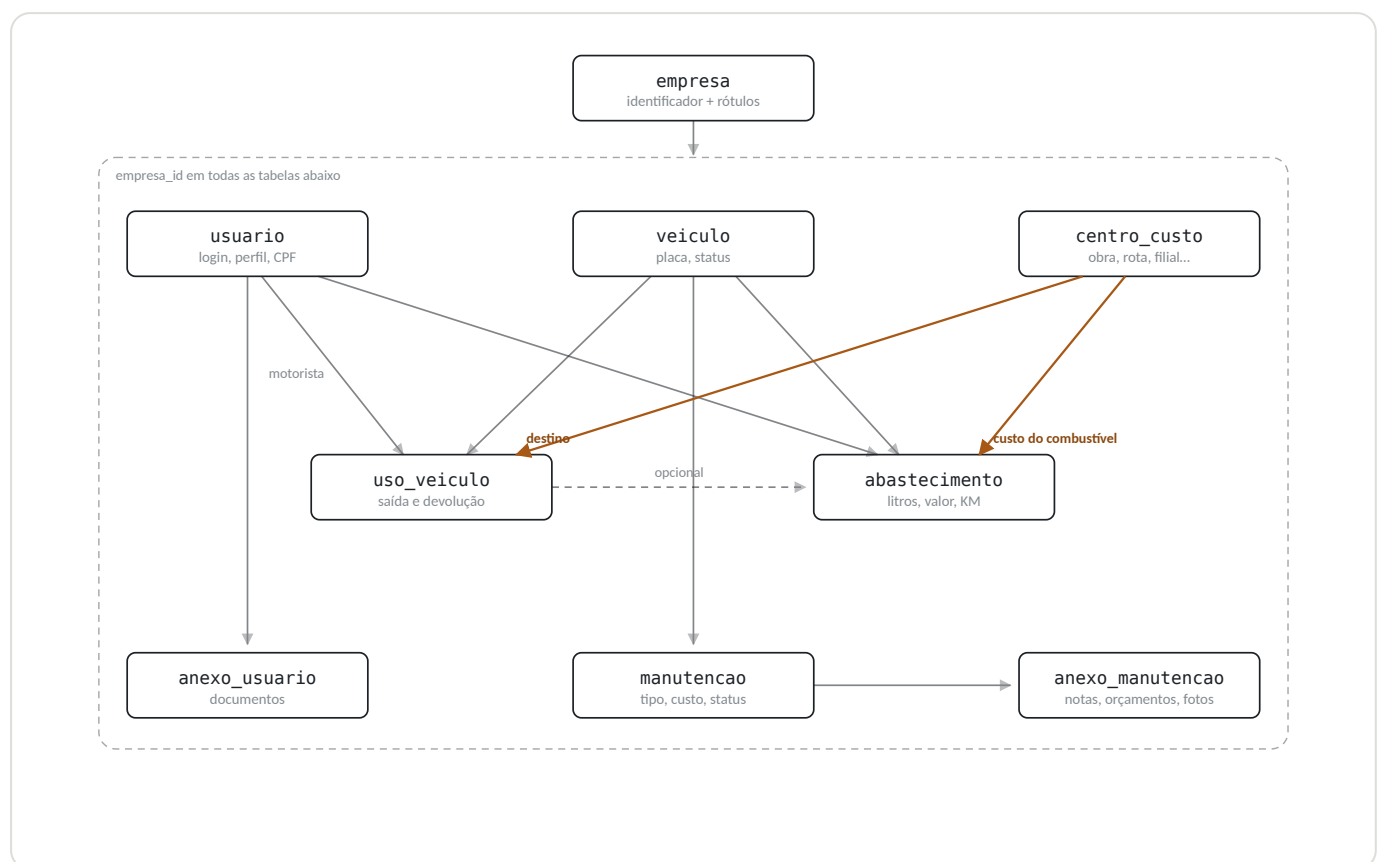
Modelo de Dados

Nove tabelas que cobrem os requisitos RF01 a RF09. Este é o mapa que vocês vão consultar enquanto codam: cada tabela vira uma classe

@Entity, cada seta vira um @ManyToOne.

Visão geral

O diagrama mostra as tabelas e como elas se ligam. Em laranja, o caminho do centro de custo — a parte que garante que o custo do veículo chegue na obra certa.



Todas as setas são relações 1:N — um veículo tem muitos abastecimentos, um centro de custo tem muitas saídas. A tabela **empresa** não recebe setas das demais: em vez disso, toda tabela carrega a coluna **empresa_id**, que é o que separa os dados de uma empresa dos da outra.

A decisão do centro de custo

A tabela **centro_custo** é genérica no banco, mas cada empresa define **como ela chama isso** nas telas. São duas colunas no cadastro da empresa, e os textos do sistema passam a usar esse rótulo em vez de palavra fixa.

A construtora cadastra o rótulo `Obra / Obras` — e todo mundo lá dentro vê "Obra de destino", "Cadastro de Obras", "Filtrar por obra". Uma transportadora no futuro cadastra `Rota / Rotas`, e uma rede de lojas cadastra `Filial / Filiais`. Mesmo banco, mesmo código, sem nenhum `if` por tipo de empresa.

Por isso os campos de endereço do centro de custo passaram a ser **opcionais**: uma obra tem endereço, um departamento não tem. Só o nome é obrigatório.

As tabelas

Tipos escritos como Java / SQL. Toda tabela tem `id` como chave primária auto-incremento e, exceto `empresa`, uma coluna `empresa_id`.

● empresa

A raiz do isolamento multiempresa. Uma linha por empresa cliente.

CAMPO	TIPO	OBSERVAÇÃO
<code>id</code>	Long / BIGINT	PK
<code>nome</code>	String(120)	obrigatório
<code>identificador</code>	String(40)	único; usado na URL de acesso da empresa
<code>rotulo_cc_singular</code>	String(40)	"Obra", "Rota", "Filial"
<code>rotulo_cc_plural</code>	String(40)	"Obras", "Rotas", "Filiais"
<code>ativo</code>	boolean	desativa a empresa sem apagar dados
<code>data_cadastro</code>	LocalDateTime	preenchido pelo sistema

● usuario

RF03. Administradores e motoristas ficam na mesma tabela, separados pelo campo perfil.

CAMPO	TIPO	OBSERVAÇÃO
id	Long	PK
empresa_id	Long	FK → empresa
nome	String(120)	obrigatório
cargo	String(80)	
cpf	String(11)	só dígitos; único dentro da empresa
login	String(60)	único dentro da empresa
senha_hash	String(60)	BCrypt; nunca a senha em texto
perfil	Enum	ADMINISTRADOR, MOTORISTA
senha_temporaria	boolean	true força troca no próximo login
tentativas_invalidas	Integer	zerado a cada login correto
bloqueado_ate	LocalDateTime	nulo quando não está bloqueado
ativo	boolean	desligamento sem apagar o histórico
data_cadastro	LocalDateTime	

● centro_custo

RF05. Era "obra" na versão 1.0. Genérica no banco, com nome de tela definido pela empresa.

CAMPO	TIPO	OBSERVAÇÃO
id	Long	PK
empresa_id	Long	FK → empresa
nome	String(120)	obrigatório
codigo	String(30)	opcional; código interno da empresa
cidade	String(80)	opcional
estado	String(2)	opcional
endereco	String(200)	opcional
ativo	boolean	obra concluída some das listas, mas fica nos relatórios
data_cadastro	LocalDateTime	

● **veiculo**

RF04. O campo status é escrito pelo sistema, não pelo usuário.

CAMPO	TIPO	OBSERVAÇÃO
id	Long	PK
empresa_id	Long	FK → empresa
nome	String(80)	ex.: "Hilux da equipe 2"
ano	Integer	
renavam	String(11)	
placa	String(8)	única dentro da empresa
tipo_combustivel	Enum	GASOLINA, ETANOL, DIESEL, FLEX, GNV, ELETRICO
status	Enum	DISPONIVEL, EM_USO, EM_MANUTENCAO, INATIVO
data_cadastro	LocalDateTime	

● **uso_veiculo**

RF06. Uma linha por saída. Devolução em branco significa que o veículo está fora agora.

CAMPO	TIPO	OBSERVAÇÃO
id	Long	PK
empresa_id	Long	FK → empresa
veiculo_id	Long	FK → veiculo
usuario_id	Long	FK → usuario (o motorista)
centro_custo_id	Long	FK → centro_custo (destino)
data_hora_saida	LocalDateTime	sugere "agora", mas é editável
data_hora_devolucao	LocalDateTime	pode ser nulo = veículo em uso
observacao	String(255)	finalidade do uso
data_registro	LocalDateTime	quando foi digitado de fato; não editável

● abastecimento

RF07. Carrega o centro de custo para que o combustível entre no custo da obra.

CAMPO	TIPO	OBSERVAÇÃO
id	Long	PK
empresa_id	Long	FK → empresa
veiculo_id	Long	FK → veiculo
usuario_id	Long	FK → usuario
centro_custo_id	Long	FK → centro_custo; sugerido pela saída em aberto
uso_veiculo_id	Long	FK opcional → uso_veiculo (rastreabilidade)
data	LocalDate	editável
quilometragem	Integer	KM no painel
litros	BigDecimal(8,3)	
valor	BigDecimal(10,2)	obrigatório
posto	String(120)	opcional
data_registro	LocalDateTime	não editável

● manutencao

RF08. Custo de manutenção pertence ao veículo, não ao centro de custo.

CAMPO	TIPO	OBSERVAÇÃO
id	Long	PK
empresa_id	Long	FK → empresa
veiculo_id	Long	FK → veiculo
tipo	Enum	PREVENTIVA, CORRETIVA
status	Enum	AGENDADA, EM_ANDAMENTO, CONCLUIDA
data_prevista	LocalDate	confirmado pelo cliente
data_conclusao	LocalDate	pode ser nulo até concluir
quilometragem	Integer	opcional; KM na data do serviço
custo	BigDecimal(10,2)	
observacoes	String(500)	
data_registro	LocalDateTime	

● anexo_usuario • anexo_manutencao

Duas tabelas com a mesma estrutura, uma para os documentos do funcionário (RF03) e outra para notas e fotos da manutenção (RF08).

CAMPO	TIPO	OBSERVAÇÃO
id	Long	PK
usuario_id ou manutencao_id	Long	FK para o dono do anexo
nome_original	String(200)	o nome que o usuário vê
nome_arquivo	String(60)	UUID gerado; é o nome real no disco
tipo_conteudo	String(80)	application/pdf, image/jpeg...
tamanho_bytes	Long	para limitar upload
data_upload	LocalDateTime	

Duas tabelas separadas em vez de uma tabela de anexos genérica. Uma tabela só exigiria referência polimórfica, que no JPA é chato e dá erro fácil. Duas tabelas quase idênticas são mais bobas e muito mais seguras de escrever.

Regras que o código precisa garantir

Toda consulta filtra por empresa

Não basta esconder o botão na tela. Toda busca por id precisa conferir se o registro pertence à empresa do usuário logado — senão, digitar `/veiculos/57` na barra de endereço acessa o veículo de outra empresa. É a regra mais importante do sistema inteiro.

O status do veículo é calculado, nunca digitado

Salvou uma saída, o veículo vira EM_USO. Preencheu a devolução, volta para DISPONIVEL. Manutenção passou para EM_ANDAMENTO, vira EM_MANUTENCAO; concluiu, volta para DISPONIVEL. Só o administrador muda para INATIVO na mão, em caso de venda ou sinistro.

Nada é apagado de verdade

Veículo, usuário e centro de custo usam o campo `ativo`. Apagar de verdade quebraria os abastecimentos e as saídas antigas que apontam para eles — e o relatório do ano passado deixaria de fechar. Inativo some das listas de seleção, mas continua nos relatórios.

Data informada e data de registro são coisas diferentes

Como os lançamentos permitem informar data retroativa, cada um guarda também o `data_registro`, que o sistema preenche e ninguém edita. Se um dia alguém questionar um lançamento, dá para saber que a saída de segunda-feira foi digitada na quarta.

Anexo não fica em pasta pública

Os arquivos vão para uma pasta fora do projeto, com nome UUID, e só são entregues por um endpoint que confere login e empresa. Se ficarem em `/static`, qualquer pessoa com o link abre sem senha.

Como a empresa é identificada no login

O cliente pediu identificação da empresa no acesso. O jeito de fazer isso sem obrigar o motorista a digitar um campo a mais é colocar a empresa no endereço, não no formulário.

Cada empresa ganha um `identificador` curto no cadastro — por exemplo `construtora-silva` — e passa a ter o seu próprio endereço de acesso:

`gestaofacil.com.br/construtora-silva`

É esse endereço que o funcionário adiciona à tela inicial do celular. Quando ele toca no ícone, o sistema **já sabe** de qual empresa ele é: ele digita só usuário e senha, como sempre. Nenhum campo novo, nenhuma explicação a dar para quem não tem intimidade com tecnologia.

Como consequência, o `login` passa a ser único **dentro de cada empresa**: a construtora pode ter um `joao` e uma futura transportadora também, sem conflito. Se alguém abrir o endereço sem a empresa, a tela mostra um campo "empresa" como alternativa — é o caminho de exceção, não o normal.

Na hora de implementar, o formulário envia o identificador da empresa junto com o usuário digitado. O jeito mais simples de resolver isso no Spring Security, sem mexer em nada exótico, é juntar os dois numa string só antes de autenticar (`joao@construtora-silva`) e buscar o usuário por essa combinação.

Decisões confirmadas com o cliente

- **Datas da manutenção:** aprovadas. `data_prevista` e `data_conclusao` entram no RF08, e é o que faz o filtro por período do RF09 funcionar para manutenções.
- **Identificação da empresa no acesso:** aprovada, pelo endereço próprio descrito acima.
- **Quilometragem:** permanece **apenas no abastecimento**. O cliente quer acompanhar o veículo, não quanto cada funcionário rodou — então saída e devolução seguem sem KM, e o modelo não muda.

Essa última decisão tem uma consequência que vale vocês saberem: sem KM na saída e na devolução, o sistema consegue dizer o custo por obra (em dinheiro), mas não os quilômetros rodados por obra. Foi escolha consciente do cliente, e está registrado assim no documento de requisitos.

Modelo de dados do Gestão Fácil, derivado dos requisitos v1.1. Quando ele for aprovado pelo grupo, vira a versão 1.2 do documento de requisitos e a base das classes `@Entity` do projeto.